



INTRODUZIONE ALLA CYBERSECURITY E AI BIGDATA

CyberSecurity

Parte 2: Crittografia, algoritmi simmetrici, funzioni di hash

Università degli Studi di Brescia

Dip. di Ingegneria dell'Informazione

A.A. 2025/2026





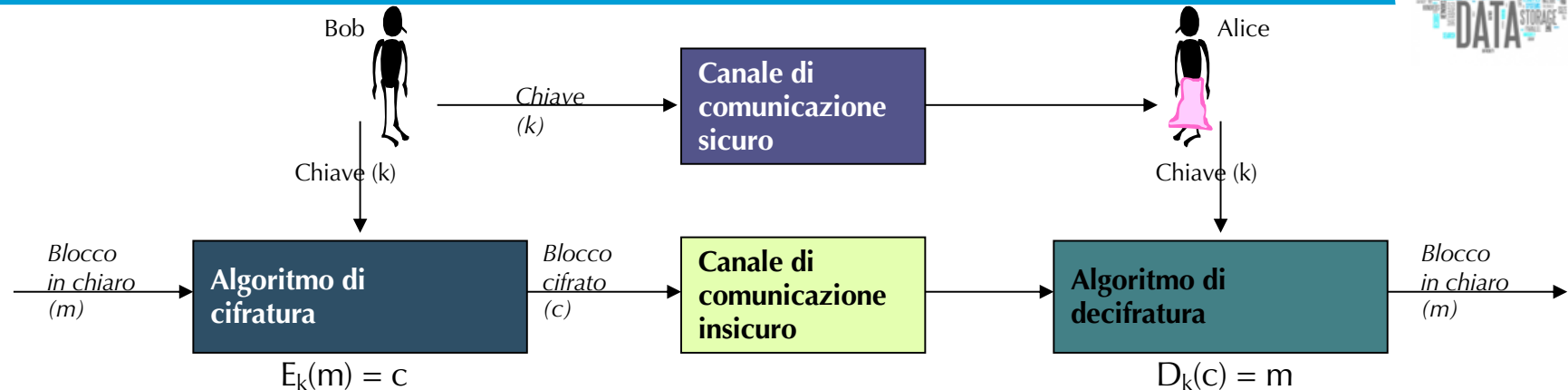
Crittografia simmetrica

Block cipher e modi d'uso



Symmetric block cipher

Schema generico ad alto livello



- Si definisce partendo dallo schema generico, dove però **la stessa chiave k viene usata sia per D che per E**
Più in generale, non è detto che debba essere necessariamente $k=k_1=k_2$, bensì, data k_1 , dev'essere molto semplice trovare k_2
- k = stringa di bit di lunghezza l
- m = stringa di bit di lunghezza b
- c = stringa di bit di lunghezza b
- Il block cipher E_k rappresenta una relazione biunivoca tra l'insieme delle stringhe m (i.e., i numeri m compresi tra 0 e 2^b-1) e i possibili 2^b valori $[0..2^b-1]$ di c , in maniera dipendente da k
- $D_k = E_k^{-1}$
- Se il messaggio x da trasmettere è più lungo di b bit, allora semplicemente si potrebbe spezzare x in una serie di blocchi m_i di lunghezza b



Gli obiettivi fondamentali di un *block cipher*



- Ogni bit di c deve dipendere da tutti i bit di k e di m
- Non ci deve essere alcuna relazione statistica evidente (almeno per chi non conosce k) tra c e m
- La modifica di un bit di m deve portare ad una modifica di ciascuno dei bit di c con probabilità 0.5
- La modifica di un bit di c deve fare in modo che il corrispondente m , ricavato con $D_k(c)$, subisca cambiamenti statisticamente non correlati con il cambiamento subito da c
- La modifica di un bit di k deve portare ad una modifica di ciascuno dei bit di c con probabilità 0.5



Gli obiettivi fondamentali di un *block cipher*



- Due classi di block cipher

Cifrario a sostituzione

Cifrario a trasposizione



La classe fondamentale dei block cipher: Il cifrario a sostituzione



- M : insieme di tutte le stringhe lunghe b bit ($|M| = 2^b$)
- Cifrario a sostituzione
 - E_k : corrispondenza biunivoca tra l'insieme finito M e se stesso
 - D_k : corrispondenza inversa
- La chiave k è un indice ad una delle possibili permutazioni su M
 - $|K|$ ha dimensione $(2^b!)$ (tutte le permutazioni di M)
 - Lunghezza chiave l bit, con $l = \log_2(2^b!) \approx b \cdot 2^b$
 - $b = 64$ allora $l = \log_2(|K|) = \log_2(2^{64}!) \approx 2^{70}$ *IMPRATICABILE!*
- Se k scelta in modo completamente casuale tra tutte le possibili chiavi è un **cifrario a sostituzione casuale e un block cipher perfetto**
 - per caratteristiche di segretezza è il limite superiore di tutti i block cipher
 - non si può fare di meglio
 - NOTA: non può fornire *segretezza perfetta* alla Shannon
 - È pur sempre un block-cipher: blocchi ricorrenti m_i allora blocchi ricorrenti c_i
 - No indipendenza statistica



Un sottoinsieme del cifrario a sostituzione: il cifrario a trasposizione



- Cifrario a trasposizione
effettua una permutazione tra tutti i bit di un dato blocco m_i
- Lo spazio delle chiavi è molto ridotto, a parità di b
Le possibili permutazioni di b bit sono $(b!)$, quindi $|K| = (b!)$
 $l = \log_2(b!)$ per $b = 64$ bit, $l \approx 384$ bit
- Segretezza: ancora peggio del cifrario a sostituzione
- TUTTAVIA questo algoritmo è utilizzabile!!



Il prodotto di cifrari



- Sostituzione e la trasposizione come blocchi fondamentali per la costruzione di una funzione E_k più complessa

Prodotto o cascata, perché rende possibili il requisito già visto:

- Modifica di un bit di m o $k \rightarrow$ modifica di ciascuno dei bit di c con $p = 0.5$
ovvero
- Modifica di un bit di $c \rightarrow m' = D_k(c')$, statisticamente non correlato a $m = D_k(c)$
- Approccio detto «**confusione e diffusione**»
output parziale permutato come input di uno step successivo
- Challenge: definire un prodotto che sia facilmente reversibile
Data la E_k , deve essere il più semplice possibile ricavare la D_k

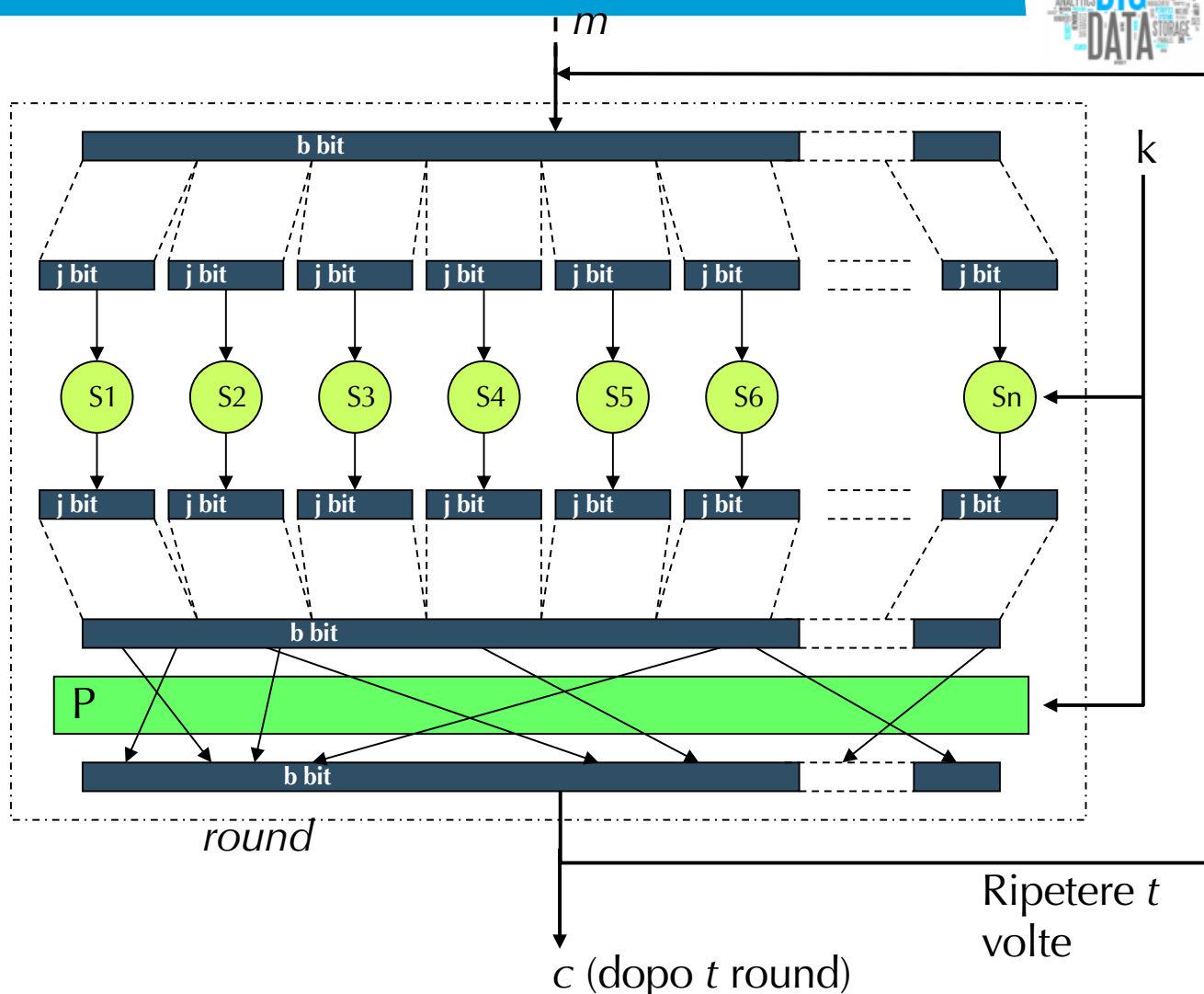


Il prodotto di cifrari: un prototipo di block cipher



- Dimensione di m e c : b bit
- $b = j \cdot n$
- S_i : cifrari a sostituzione che operano su j bit
- P : cifrario a trasposizione che opera su b bit
- *Round*: prodotto dei cifrari
- Sono necessari t round, con $t > 1$, perché si raggiunga l'obiettivo fondamentale

cambiamento di un bit di m porta al cambiamento di *ciascuno* dei bit di c con probabilità 0.5



Prodotto di cifrari: note



- Prodotto di cifrari: ***combinazione di funzioni elementari*** (swap, bit-shift, separazioni, concatenazioni, XOR, substitution-cipher, ...)
- Il dimensionamento ottimale del numero di round è uno dei parametri di progetto più importanti
- Come utilizzare k in ciascuno dei cifrari utilizzati nella combinazione?
- Come assicurare la reversibilità?



Il *Data Encryption Standard* (DES)



- Cifrario simmetrico a blocchi
 - $b = 64$ bit
 - $l = 56$ bit (64 in teoria; 56 in pratica [vedere più avanti])
- Veloce nelle implementazioni HW, relativamente lento nelle implementazioni SW
- Nasce da un progetto commissionato dalla USA *National Security Agency* (NSA) ad IBM
 - In pratica, il progetto fu gestito da IBM, su commissione e con direttive NSA
- Pubblicato nel 1975 ed in seguito standardizzato nel 1977 dalla USA *National Institute for Standards and Technology* (NIST), per uso **commerciale** e per applicazioni governative **non-classified**
- In pratica il più vecchio e il più usato cifrario a blocchi commerciale (conosciuto) ancora in uso oggi, seppure con metodologie di applicazione più complesse rispetto a quanto si facesse tempo fa (cfr. triple-DES, più avanti)



DES: storia e folklore

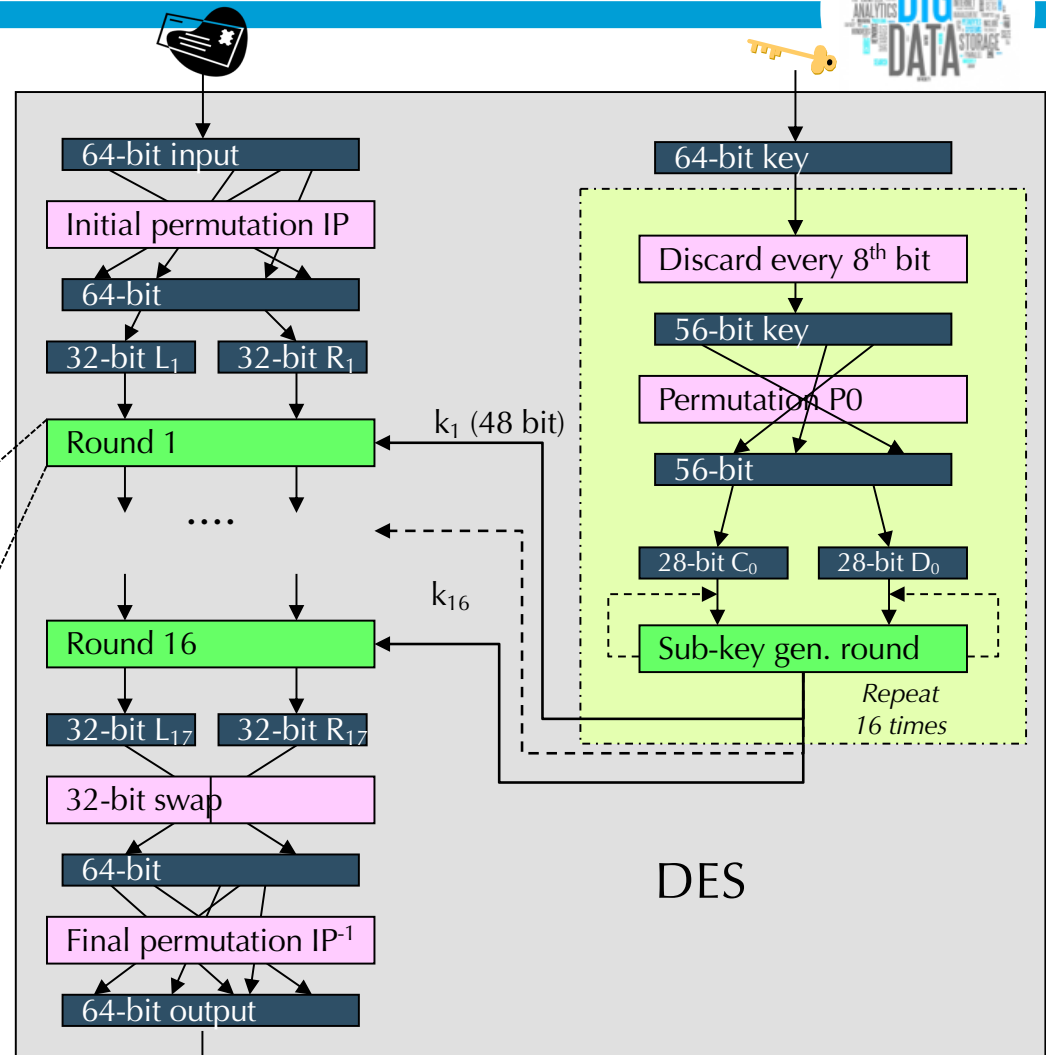
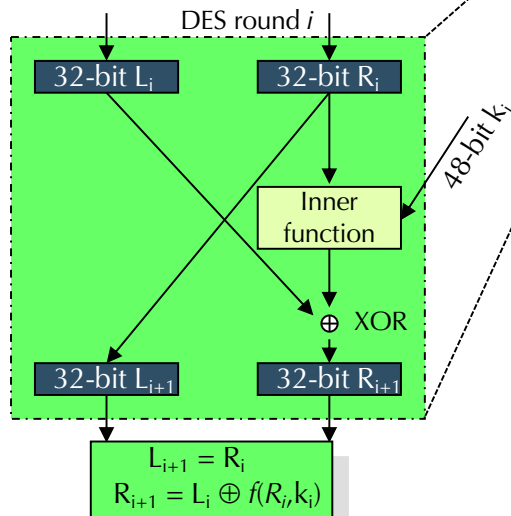


- I parametri e i requisiti di progetto delle parti cruciali di DES sono rimasti segreti per molto tempo: molte delle analisi che faremo sono estrapolati dall'algoritmo stesso
 - Motivazione per la segretezza: DES fu progettato per resistere ad attacchi crittanalitici che all'epoca erano pressoché sconosciuti dalla comunità scientifica
 - Problema: dubbi sull'esistenza di *trap-door*
- Implementazione in HW molto più semplice rispetto all'implementazione in SW
 - L'analisi di alcuni dei parametri scelti (per esempio le permutazioni iniziali e finali) mostra come, probabilmente, l'algoritmo fu progettato con l'esplicito obiettivo di renderne l'implementazione in SW molto meno efficiente dell'implementazione in HW
 - DES è anche relativamente complicato da implementare in SW a causa del fatto che molte sue operazioni elementari sono orientate ai bit, non ai byte
- Molti sostengono che la standardizzazione di DES con una chiave lunga soli 56 bit non fu un fatto involontario
 - È legittimo pensare che l'NSA volesse standardizzare un algoritmo che fosse robusto a sufficienza per le applicazioni commerciali, ma che fosse debole quanto bastasse perché almeno loro potessero violarlo con una certa facilità



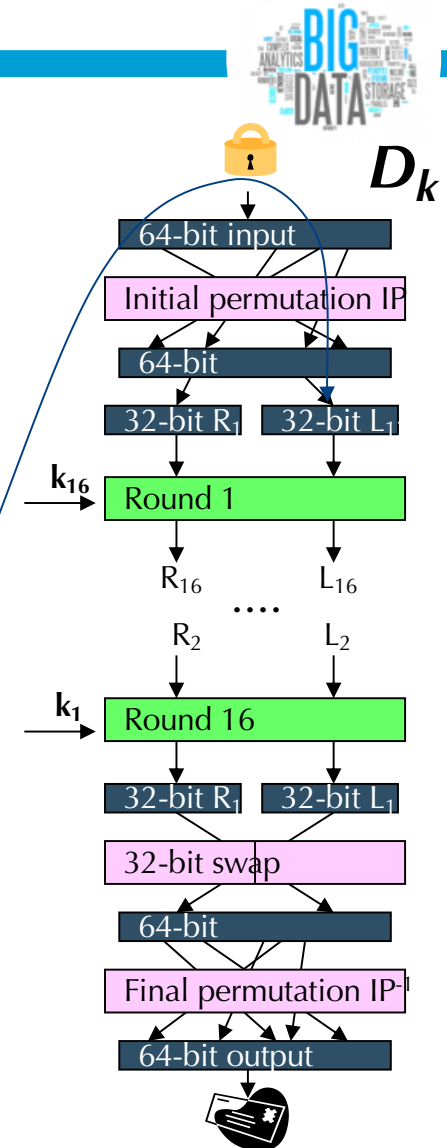
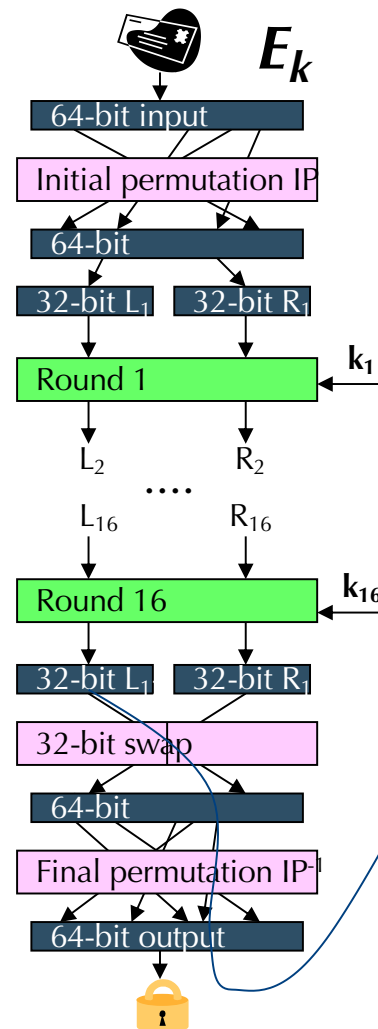
DES: schema ad alto livello

- Permutazioni indipendenti dalla chiave
- «Inner function»
Look-up table, unica parte non lineare
- Prodotto di cifrari:
confusione e diffusione tramite round!
ogni round usa sottochiave k_i
 - sottochiavi generate dalla chiave principale
- cifrario di tipo *Feistel* (due metà)
 - sinistra (L) - destra (R)



DES: encryption vs decryption

- Lo stesso "circuito" per crittare e decrittare
- Cambia solamente l'ordine di applicazione delle sottochiavi!



DES: analisi ad alto livello

Le chiavi



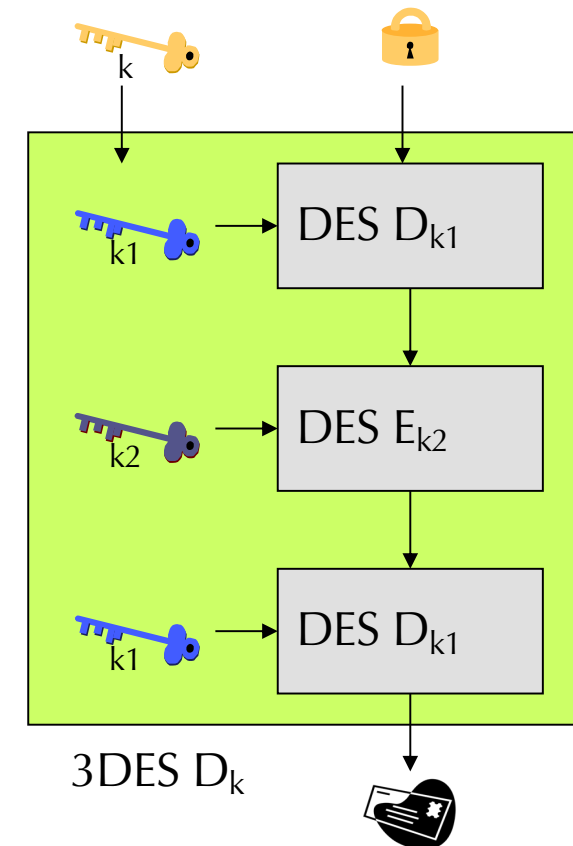
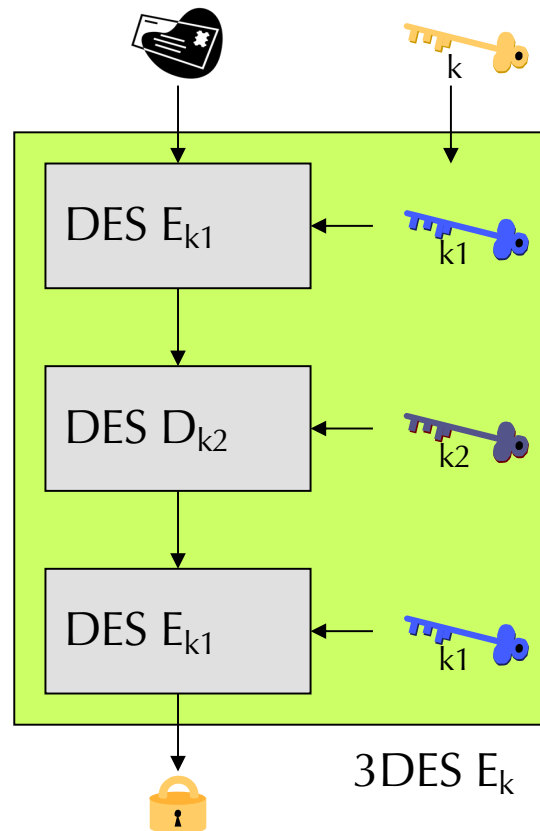
- Solo 56 dei 64 bit della chiave vengono effettivamente usati
Ogni ottavo bit viene scartato, è bit di parità per verifica della chiave
Attacco forza bruta $\sim 2^{56}$ iterazioni (ma non è vero! vedi prossima riga)
- Proprietà di complementarietà: se $c = E_k[m]$ allora $c' = E_{k'}[m']$
Attacco forza bruta $\sim 2^{55}$ iterazioni in realtà
- 1977: macchina US government - attacco forza bruta in 12 ore, costo 20M\$ (del tempo)
- 1993: attacco forza bruta in meno di 3 ore e mezza, costo 1M\$ (del tempo)
57600 chip DES, ciascuno in grado di testare 1 chiave ogni 20 ns
- 2007: COPACOBANA (cost-optimized parallel code breaker)
circa 8K\$ (del tempo)
integra un centinaio di FPGA, sfrutta varie tecniche crittanalitiche
brute-force di DES in meno di una settimana
- 2022: C. Tezcan, "Key lengths revisited: GPU-based brute force cryptanalysis of DES, 3DES, and PRESENT," Journal of Systems Architecture 124.
Come COPACOBANA ma con 13 RTX 3070 GPU ($13 * 480\$ \sim 6000\$$)



Il DES oggi: 3DES



- DES oggi non è ritenuto sicuro
chiave corta
tecniche crittanalitiche avanzate
sono ordini di grandezza migliori
degli attacchi a forza bruta su
DES
- Si usa 3DES:
 $b = 64$ bit
 $k = 112$ bit
 $k = k_1/k_2$
- Spazio delle chiavi è 2^{112}
- Retrocompatibile con DES:
basta porre $k_1 = k_2$



DES: conclusioni



- **Vantaggi di DES**

Omnipresente: standard vecchio ne ha permesso una diffusione capillare

È l'algoritmo ancora in uso più crittanalizzato: ciononostante DES ha rivelato un ottimo progetto di base, e in molti casi pratici, l'attacco a forza bruta (ricerca esaustiva su metà dello spazio delle chiavi) rimane l'attacco migliore a tutt'oggi

- **Svantaggi di DES**

Chiave troppo piccola!

Relativamente lento

Il sospetto sulle trap-door dell'NSA, sebbene ormai non più molto ragionevole

- **Perché rimpiazzare DES?**

Difficoltà di implementazione in SW o HW general purpose (operazioni orientate ai bit, non ai byte)

Chiave corta, a meno di usare 3DES, che però è relativamente lento

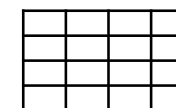
Disponibilità di nuovi strumenti matematici per nuovi cipher (per es.: campi finiti)



Advanced Encryption Standard (AES)



- Algoritmo a blocchi, standardizzato come successore di DES nel 2001, basato su un algoritmo (Rijndael) introdotto nel 1999, *royalty-free*
 - Processo di standardizzazione aperto (concorso pubblico, con regole dettate dal NIST)
 - Dimensione blocco $b = 128$ bit
 - l = variabile, 128, 192, 256 bit (AES-128, AES-192, AES-256)
- Sviluppato da J. Daemen e V. Rijmen
 - presentarono al concorso l'algoritmo Rijndael
- Basato sulla teoria matematica dei *campi finiti* (di Galois)
- Opera su blocchi $N \times N$, detti stati
 - versione a 128bit, stati = matrici di 4×4 byte ($4 * 4 * 8\text{bit} = 128\text{bit}$)
- Usa round, sottochiavi per round, blocchi tipo S-BOX
 - round e sottochiavi $\rightarrow$ confusione diffusione
 - S-BOX $\rightarrow$ non-linearità
- Numero di round dipende alla lunghezza della chiave scelta!



AES-128: blocchi 4x4



- Stringa «bigdata cybersec»:

blocco di testo in chiaro

b	i	g	d	a	t	a		c	y	b	e	r	s	e	c
62	69	67	64	61	74	61	20	63	79	62	65	72	73	65	63

stato
(iniziale)

62	61	63	72
69	74	79	73
67	61	62	65
64	20	65	63



AES



k_1 = «cybersecurity»



07	3d	ad	22
75	d0	16	16
b2	36	2b	db
b9	57	87	b0

stato
(finale)

07	75	b2	b9	3d	d0	36	57	ad	16	2b	87	22	16	db	b0
.	u	.	.	=	.	6	W	.	.	+	.	«	.	.	.

blocco di testo cifrato



AES: operazioni e round



- 4 tipi di operazioni sugli stati:

SubBytes

- ogni byte sostituito mediante S-box, parte non lineare

ShiftRows

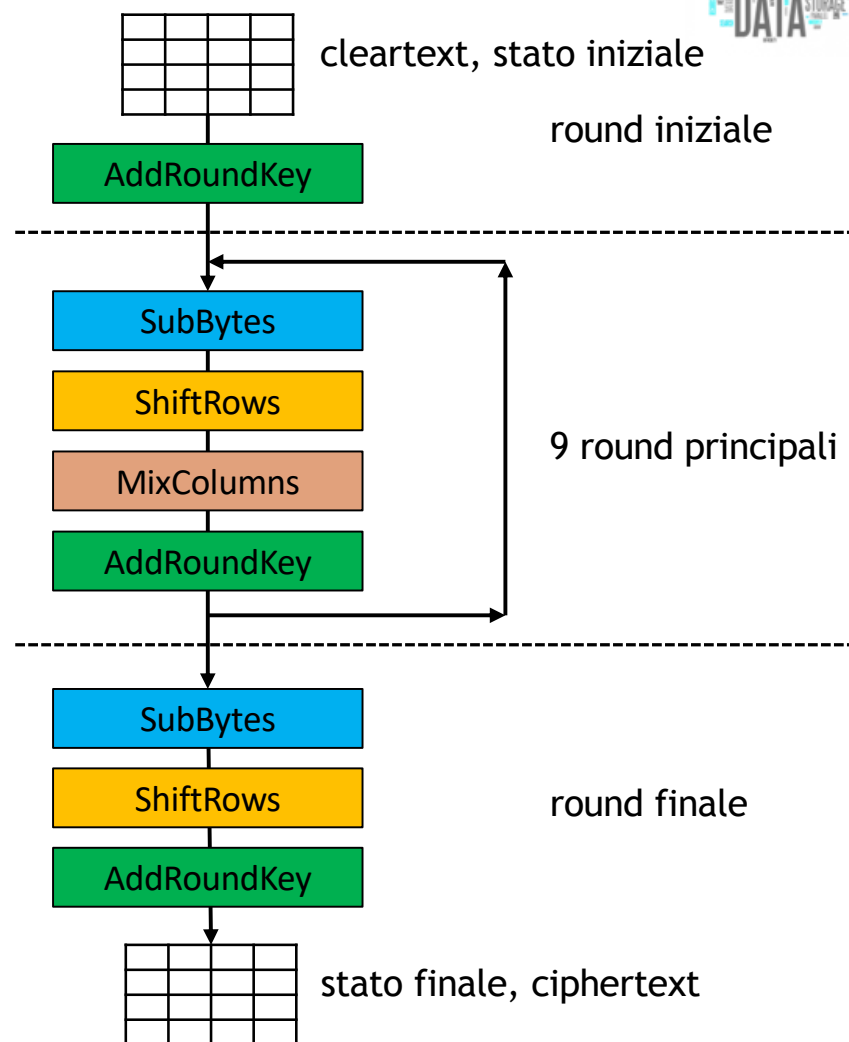
- ogni riga viene ruotata di una certa quantità di posizioni

MixColumns

- ogni colonna viene moltiplicata per una matrice e origina nuova colonna

AddRoundKey

- lo stato è combinato con la chiave del round



AES: note conclusive



- Progettato per essere semplice da implementare sia in HW che in SW
in entrambi i casi si ottiene un algoritmo molto più veloce di DES
- Non si conoscono attacchi crittanalitici pratici più efficienti della forza bruta
- Diversamente da DES non è una rete di Feistel
implementa una rete a sostituzione e permutazione
come DES verifica il principio di confusione e diffusione
- Richiede poca memoria
- Secondo NSA AES fornisce
livello «SECRET» con chiavi da 128bit
livello «TOP-SECRET» con chiavi da 256bit
- Pochi attacchi pubblicati (XSL, 2002; best brute-force, 2009)
formalmente corretti ma con tempi di esecuzione che non li rendono comunque praticabili



International Data Encryption Algorithm (IDEA)



- Algoritmo a blocchi, introdotto nel 1991, coperto da brevetto
 - $b = 64$: sufficiente, ad oggi, per proteggere da attacchi basati su statistiche del ciphertext
 - $l = 128$: sufficiente, ad oggi, per proteggere da ricerche esaustive nello spazio delle chiavi
- Progettato per rispondere alle esigenze base degli algoritmi a blocchi, e per resistere a tecniche crittanalitiche avanzate (es., differential cryptanalysis)
- Progettato per facilitare implementazioni efficienti:
 - In HW: regolarità delle strutture, minime differenze tra E e D
 - In SW: operazioni orientate ai multipli degli ottetti (16 bit, nel caso di IDEA), operazioni elementari facilmente realizzabili a partire da funzioni base disponibili su ogni processore general purpose (bit-shift, somma, ecc.)
- Soddisfa tutti questi requisiti: nessuno, finora, ha **pubblicato** alcun metodo pratico (al di là degli attacchi a forza bruta) per penetrare IDEA
- Idea viene usato in diversi prodotti e sistemi commerciali: PGP, S/MIME, ...
- Crittanalizzato ampiamente dalla comunità, ma “solo” per una trentina d’anni, senza che sia stato trovato alcun problema
- Qualche voce lo da per *breakable* dall’NSA, ma sono solo voci...



Blowfish (BF)



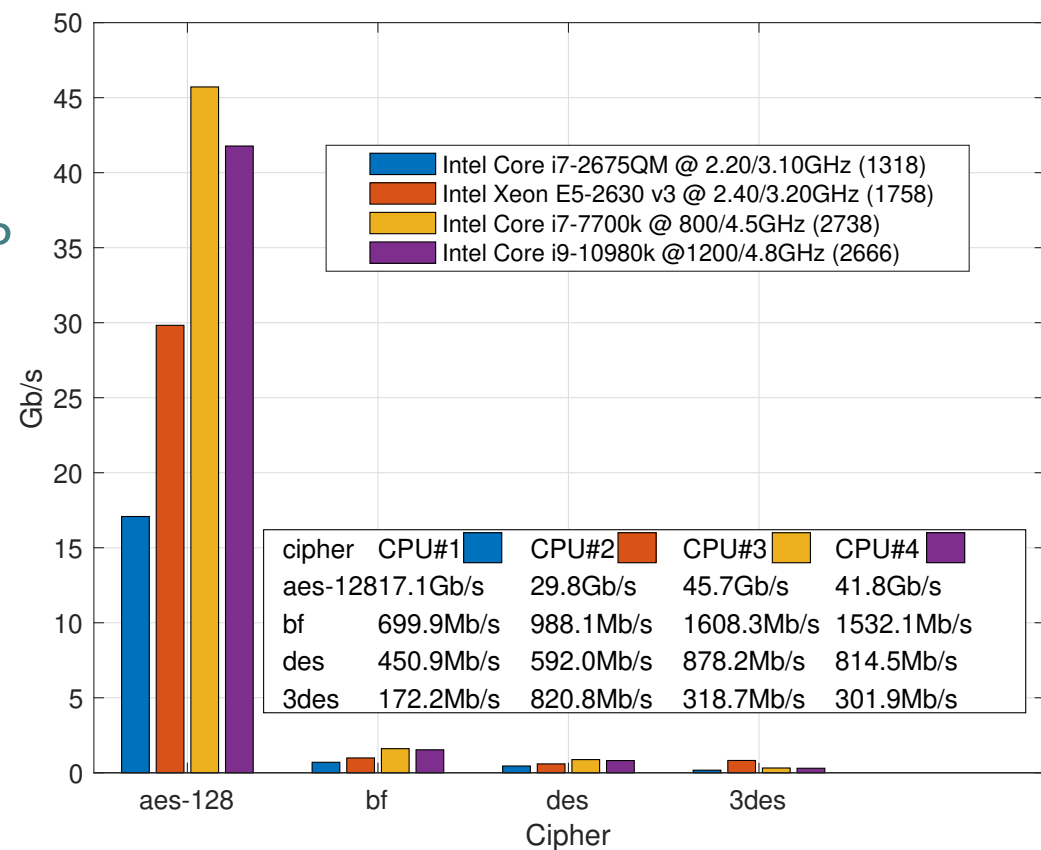
- Algoritmo a blocchi, introdotto nel 1994, *royalty free*
 - $b = 64$ bit
 - $l =$ variabile, da 32 a 448 bit
- Uno dei primi algoritmi ancora in uso ad introdurre il concetto di *sicurezza variabile*, legato alla variabilità della dimensione della chiave
- Progettato per essere estremamente efficiente in implementazioni SW su processori general-purpose
- È molto meno efficace in implementazioni HW: problemi con l'algoritmo di generazione delle sotto-chiavi (estremamente complesso, e richiede “molta” memoria)
- Utilizza funzioni elementari combinate in maniera semplice, il che lo rende relativamente semplice sia da analizzare che da implementare
- Implementato in vari meccanismi di sicurezza (esempio: OpenSSH, PGPfone, ...)
- Ha ricevuto ancora meno attenzione (temporale) di IDEA, ma finora non sono stati trovati problemi seri



Performance dei vari block-cipher



- Test fatti “in casa”
creato grande file in ramdisk
crittato più volte
risultato mediato
- Note:
IDEA manca per ragioni di IP





Crittografia simmetrica

Stream cipher e

Modi d'uso dei Block cipher

(cenni)



Stream cipher vs. block cipher



- Block cipher
 - Processano il *plaintext* in blocchi “larghi”,
 - i.e. normalmente $b \geq 64$ bit
 - E_k è *stateless* (o *senza memoria*): la stessa E_k viene usata per crittografare tutti i blocchi di plaintext
- Stream cipher
 - Processano il *plaintext* in blocchi “piccoli”,
 - i.e. normalmente $1 \leq b \leq 8$
 - E_k è, in genere, *stateful* (con memoria):
 - essa varia man mano che trasforma il plaintext
- NOTA: è possibile trasformare i block cipher in stream cipher!
Modi d'uso dei block cipher



Stream cipher: one-time-pad vs. pseudo-random keystream



- Abbiamo già visto one-time-pad
 - Unico esempio di segretezza perfetta
 - stream cipher perfetto!
- Possiamo avvicinarci generando un *keystream* lungo quanto il messaggio a partire da una chiave k "corta"
 - Il keystream in questo caso non è perfettamente casuale
 - è *pseudocasuale*
 - Quindi... non può esserci perfetta segretezza
 - $H(k) \ll H(m)$

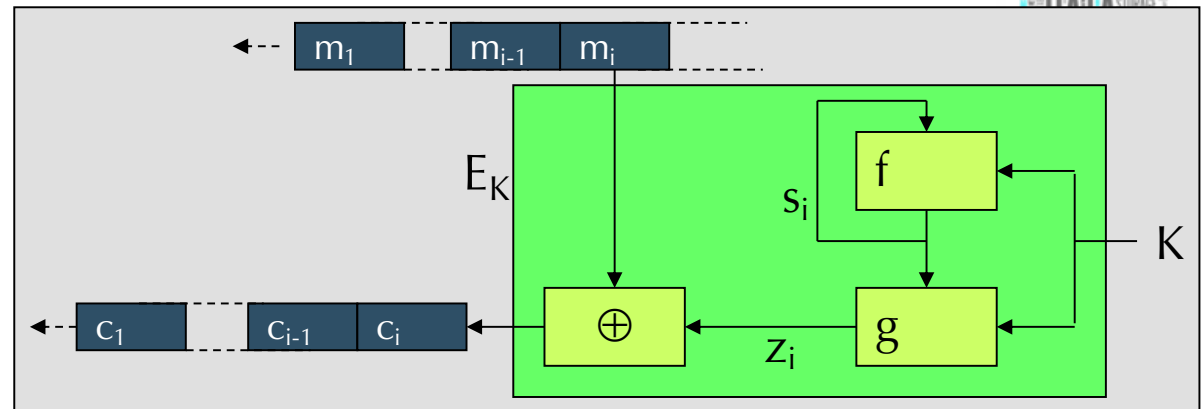


Synchronous stream cipher



Encryption

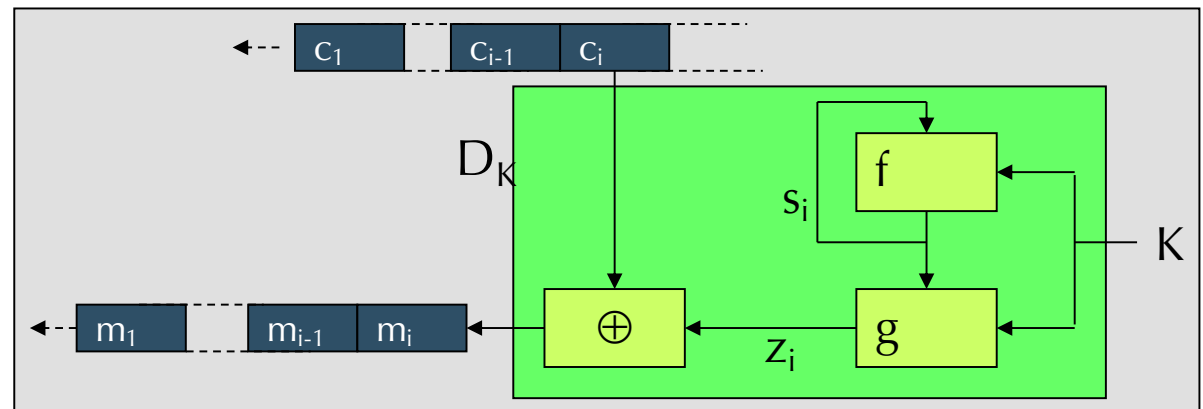
$$\begin{aligned}c_i &= m_i \oplus z_i \\ z_i &= g(s_i, K) \\ s_{i+1} &= f(s_i, K)\end{aligned}$$



Siamo lontani dalla segretezza perfetta: $H(k) \ll H(m)$

Decryption

$$\begin{aligned}m_i &= c_i \oplus z_i \\ z_i &= g(s_i, K) \\ s_{i+1} &= f(s_i, K)\end{aligned}$$



Synchronous stream cipher: proprietà principali



- La concatenazione dei vari z_i porta ad un keystream pseudocasuale
Cifratura avviene operando lo XOR del clear-text con il keystream
- Sorgente e destinazione devono **sincronizzare**
il flusso di dati
il processo di generazione e uso delle z_i
- Perdita/aggiunta di un solo blocco: decifrazione errata dei seguenti!
- Necessari meccanismi per limitare gli effetti di questi
marker, re-inizializzazione del processo, ecc.
- Errori (di trasmissione) in un blocco non portano errori nei blocchi successivi
- **Non propagano l'errore**
si adattano bene alla trasmissione su canali con un alto tasso d'errore

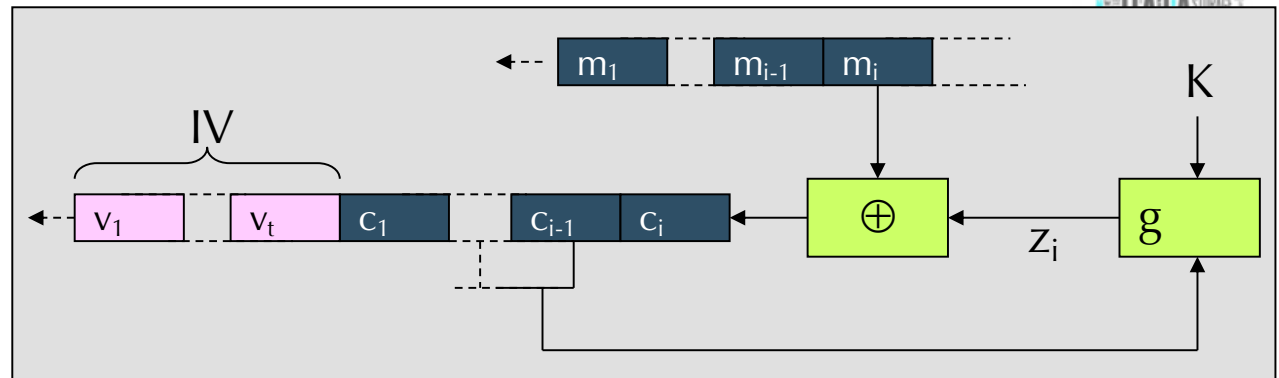


Self-synchronizing stream cipher



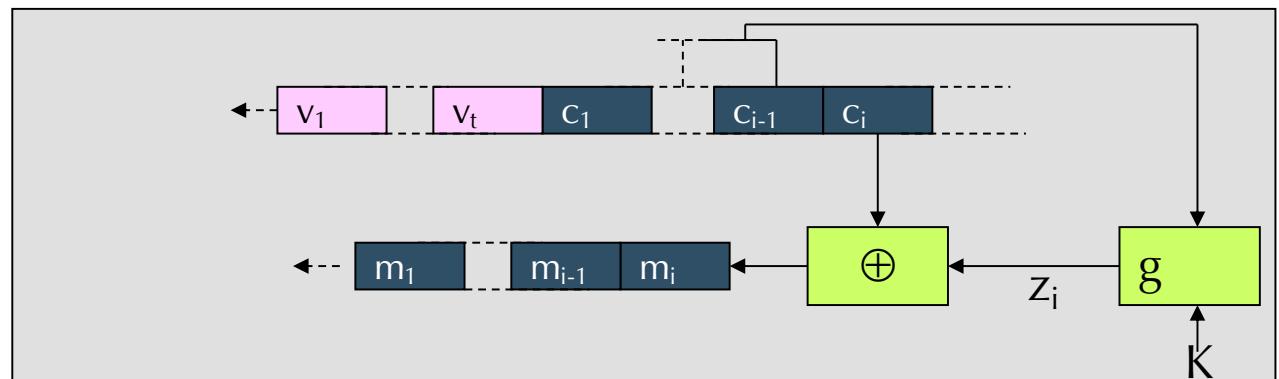
Encryption

$$\begin{aligned} c_i &= m_i \oplus z_i \\ z_i &= g(c_{i-1}, \dots, c_{i-t}, K) \\ \{c_{-t}, c_{-t+1}, \dots, c_0\} &= \{v_1, v_2, \dots, v_t\} \end{aligned}$$



Decryption

$$\begin{aligned} m_i &= c_i \oplus z_i \\ z_i &= g(c_{i-1}, \dots, c_{i-t}, K) \\ \{c_{-t}, c_{-t+1}, \dots, c_0\} &= \{v_1, v_2, \dots, v_t\} \end{aligned}$$



Self-synchronizing stream cipher: proprietà principali



- Concatenazione dei z_i porta a keystream pseudocasuale
Cifratura: si applica lo XOR tra clear-text e keystream
- Initialization Vector (IV): serie di blocchi iniziali non segreti
necessari per inizializzare il processo di generazione degli z_i
- **Self-synchronizing**: cancellazione di un c_i , l'inserimento di un c_i "spurio", o gli errori di trasmissione su un certo c_i portano ad errori di decrittazione solo su un limitato numero di m_j che seguono l' m_i corrotto

La quantità di m_j corrotti dipenderà dalla grandezza di t



RC4: uno degli stream cipher più usati



- È un synchronous stream cipher, creato da Ron Rivest (la “R” di RSA) nel 1987
 - Protetto da segreto fino al 1994, quando il codice sorgente fu illegalmente inviato in rete
- Genera un keystream a partire da una chiave K lunga tra 1 e 256 byte
 - I primi 256 byte del keystream mostrano correlazione alta con la chiave
 - Da scartare
- Estremamente semplice da implementare, veloce ed efficiente sia in HW che in SW
 - RC4 nelle implementazioni commerciali è circa 10 volte più veloce di DES
- In via di abbandono... (es. Wi-Fi non lo usa più)



Chacha-20



- Derivato da Salsa-20, finalista della competizione eSTREAM (2005)
Progetto europeo: goal era creazione di nuovi stream-cipher (per sw e hw)
-20: numero di passaggi intermedi!
Requisiti di Salsa-20: uso di operazioni semplici e quindi grande velocità
 - Ad esempio, diversamente da DES/AES non richiede pre-computazioni di chiavi
- Chacha-20:
riprende schema di Salsa-20: usa operazioni di XOR, somma(!) e rotazioni
-20 passaggi intermedi su uno stato dato da 16 valori interi a 32 bit (4x4)
chiavi da 256-bit, nonce di inizializzazione (è uno stream-cipher) di 128-bit
Miglior attacco: key recovery ha complessità di 2^{236}



Modi d'uso dei Block cipher

Ovvero: dai Block cipher agli Stream cipher



- *Modi d'uso* dei block cipher
 - quei meccanismi che permettono l'uso dei Block cipher con messaggi più lunghi del block size
- (Quasi) Tutti i modi d'uso trasformano un Block cipher in uno Stream cipher, a volte synchronous, altre volte no
 - vari modi d'uso possiedono vantaggi e svantaggi
 - si scelgono in funzione dell'applicazione
- Nota: modi d'uso non legati alla crittografia simmetrica possono essere usati anche per la crittografia asimmetrica!

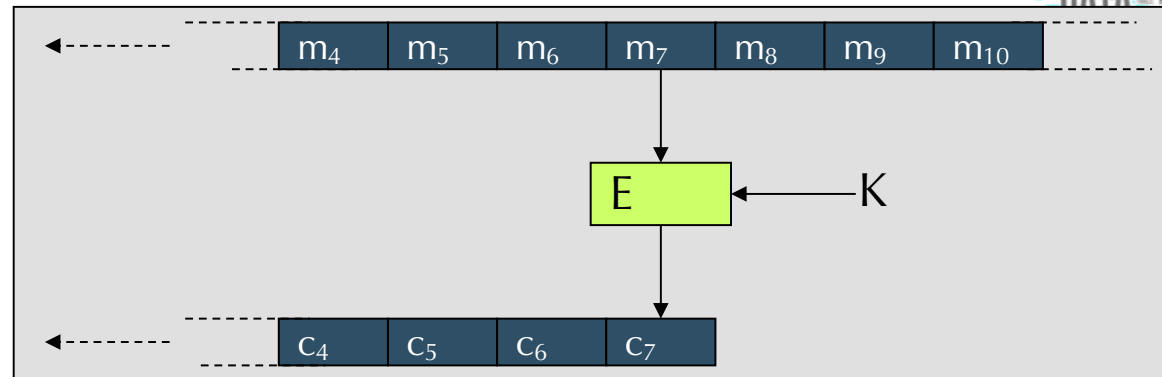


Electronic Code Book mode (ECB)



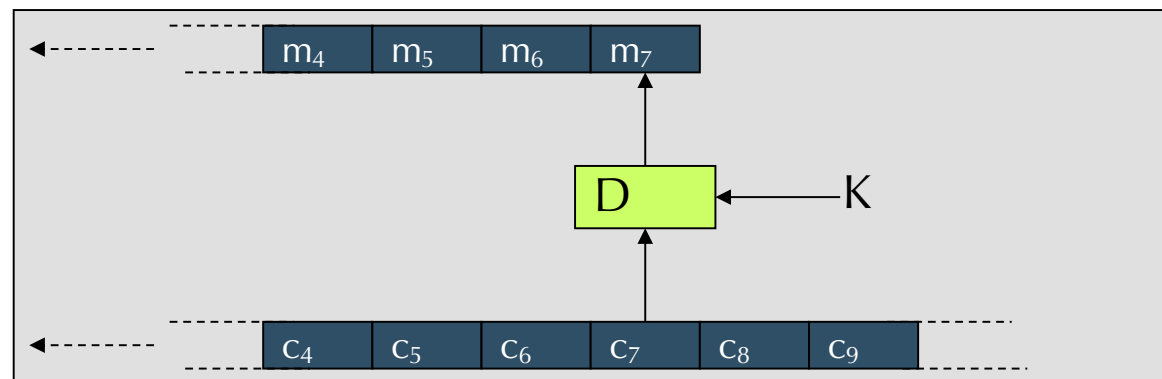
Encryption

$$c_i = E_k(m_i) \\ 1 \leq i \leq n$$



Decryption

$$m_i = D_k(c_i) \\ 1 \leq i \leq n$$



ECB: proprietà



- Vantaggi

- Semplicissimo da implementare

- Non propagazione degli errori, per errori di tipo “cambio di valore di bit” (*bit error*)

- Svantaggi

- Si presta ad attacchi (passivi) che analizzano la frequenza dei simboli

- Si presta ad attacchi (attivi) che manipolano l'ordine dei blocchi, o ne modificano il valore (violazione dell'integrità e autenticità)

- Propagazione degli errori, per errori del tipo “inserimento o cancellazione di bit” (*lost/inserted bit*)

- A causa di questi svantaggi, è bene ***non usare mai ECB***

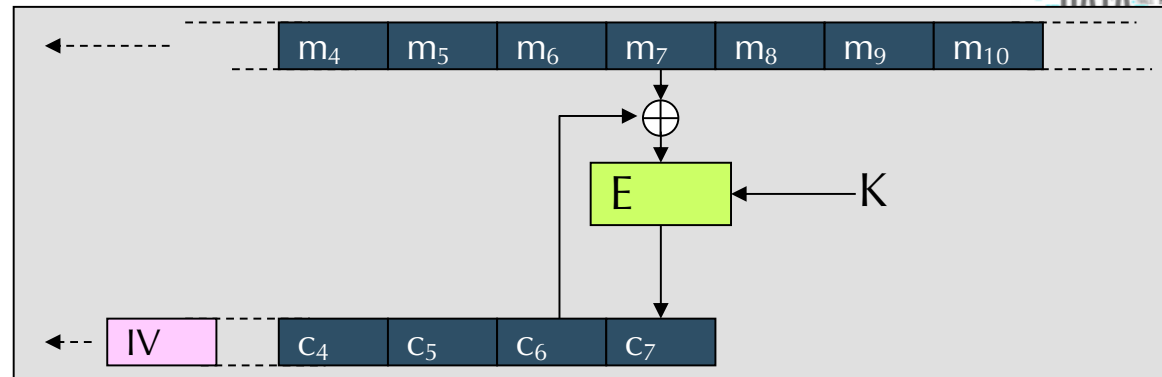


Cipher Block Chaining mode (CBC)



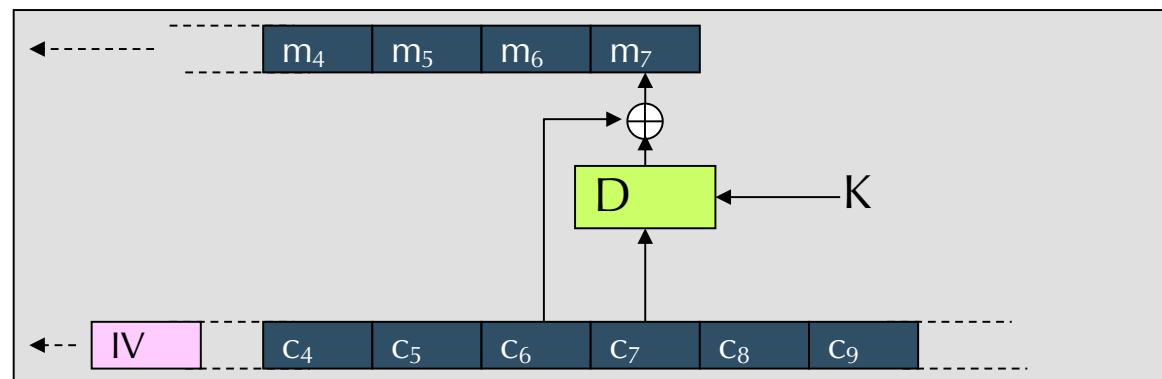
Encryption

$$c_i = E_k(m_i \oplus c_{i-1})$$
$$1 \leq i \leq n$$
$$c_0 = IV$$



Decryption

$$m_i = D_k(c_i) \oplus c_{i-1}$$
$$1 \leq i \leq n$$
$$c_0 = IV$$



CBC: proprietà



- Vantaggi

Non propaga gli errori: un singolo errore su c_i agisce su m_i e m_{i+1}

Stessi valori di m in posizioni diverse producono diversi c :

- molto più resistente di ECB all'analisi della frequenza dei simboli

- Svantaggi

Si presta ad attacchi attivi che **aggiungono blocchi**, o che modificano il valore di un blocco c per modificare in maniera prevedibile il corrispondente m (violazione di integrità e autenticità)

- Nota: importanza della presenza dell'IV, e della sua casualità

Esempio: se l'IV rimane costante, la trasmissione periodica dello stesso messaggio porterà allo stesso ciphertext, quindi un intruso potrà capire **quando** il messaggio è cambiato

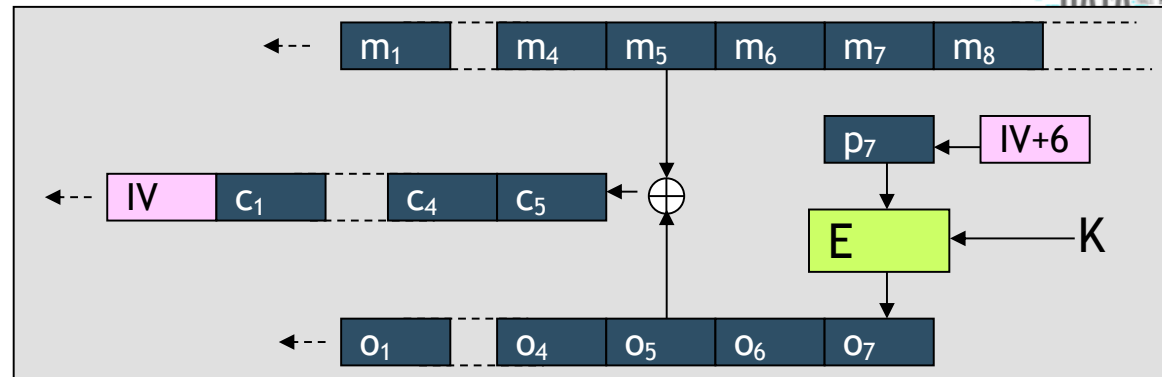


Counter mode (CTR)



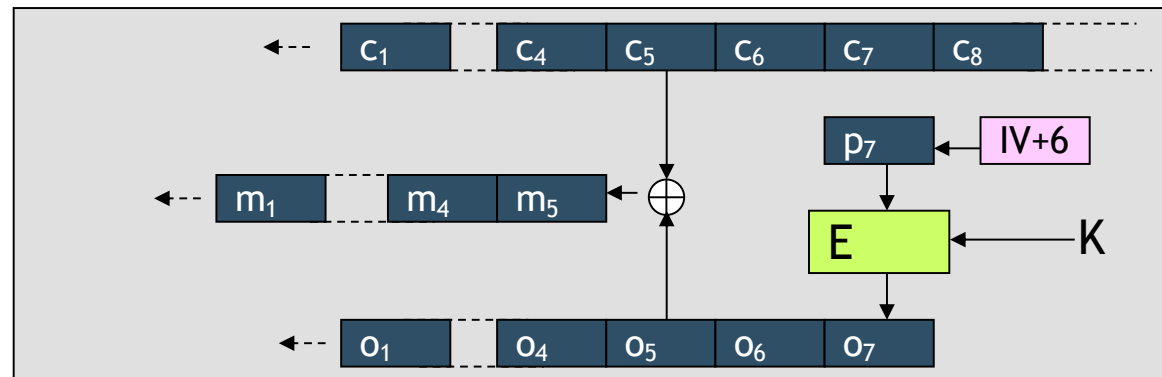
Encryption

$$\begin{aligned}c_i &= o_i \oplus m_i \\ o_i &= E_k(p_i) \\ p_i &= IV + i - 1 \\ 1 \leq i \leq n\end{aligned}$$



Decryption

$$\begin{aligned}m_i &= o_i \oplus c_i \\ o_i &= E_k(p_i) \\ p_i &= IV + i - 1 \\ 1 \leq i \leq n\end{aligned}$$



CTR: proprietà



- Vantaggio principale: possibilità di calcolare il keystream necessario alla cifratura/decifratura del blocco j -esimo, senza dover calcolare o avere a disposizione tutti i $(j-1)$ blocchi che lo precedono
- Questo è utile, e a volte necessario, per la crittografia di dati che devono essere *random access* (file, dischi, ecc.)





Funzioni di hash



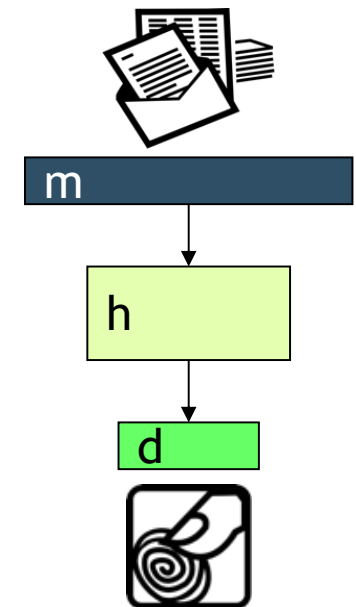
Funzioni di hash: definizioni



- Una (*one-way*) *hash function* h (anche detta *message digest function*), è una funzione che realizza una corrispondenza **univoca** tra un insieme M di stringhe binarie di lunghezza arbitraria, e un insieme D di stringhe binarie, ciascuna di lunghezza n fissata (la dimensione dell'hash)
- Il valore $d = h(m)$ può essere visto come una rappresentazione compatta di m
- Un esempio d'uso

Pubblicare un messaggio e il suo *hash*

- Esempio: <http://releases.ubuntu.com/bionic/>
- scorrere fino a MD5SUMS (MD5 è uno dei possibili hash)
`72491db7ef6f3cd4b085b9fe1f232345 *ubuntu-18.04.3-desktop-amd64.iso`
`c038a031a2b638f8e89d897119f1b7bb *ubuntu-18.04.3-live-server-amd64.iso`
- scaricare l'immagine e calcolare l'hash MD5
 - A cosa serve?



Funzioni di hash: a che servono?

Funzioni di hash con e senza chiave



- Senza chiave: *Modification Detection Code* (MDC)

Esattamente come le abbiamo definite prima: $d=h(m)$

Obiettivo: ipotizzando di poter garantire l'integrità di d , la $h(m)$ può garantire l'integrità di m (d è il *fingerprint* di m)

Attacchi possibili (e, ortogonalmente, grado di sicurezza di un MDC)

- Data $d=h(m)$, trovare m' t.c. $h(m')=d$
- Trovare m e m' t.c. $h(m)=h(m')$

- Con chiave: *Message Authentication Code* (MAC)

In questo caso $d=h_k(m)$

Obiettivo: autenticazione e integrità di m

Attacchi possibili (e, ortogonalmente, grado di sicurezza di un MAC)

- Osservando un numero sufficiente di d , scoprire la k usata per generarli
- Osservando un numero sufficiente di $d_i=h_k(m_i)$, calcolare il valore $d'=h_k(m')$, senza conoscere k
- Trovare m e m' tale che $d=d'$, dove $d=h_k(m)$ e $d'=h_k(m')$

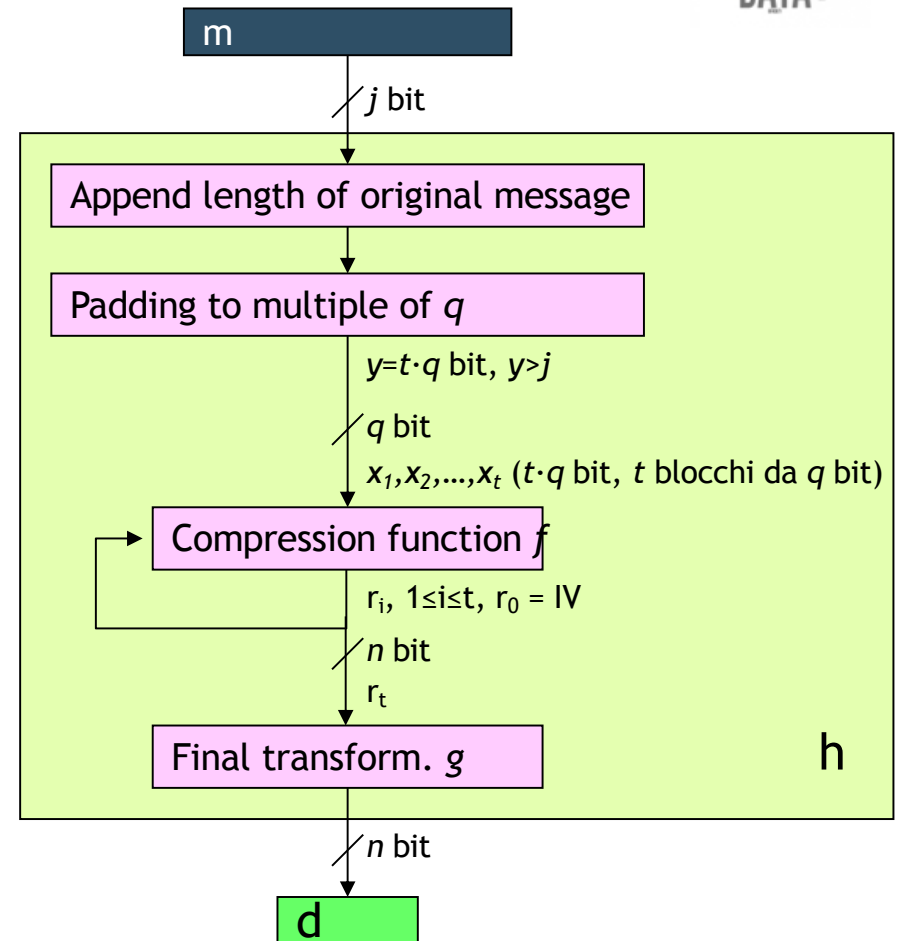


MDC

Modello base: funzioni di hash iterative



- Fase preparatoria
 - Si concatena il messaggio originale con un numero binario che indica la lunghezza del messaggio stesso
 - Padding, per ottenere, da una lunghezza originaria del messaggio arbitraria (j), un multiplo del blocco base q sul quale opera la funzione di compressione f
- La funzione di compressione f accetta come input un blocco di q bit (blocco del messaggio originale) più una variabile di stato di n bit, e produce un valore di n bit
- t iterazioni della funzione di compressione, con il risultato intermedio $r_i = f(x_i, r_{i-1})$, $r_0 = IV$
 - IV può essere fisso, funzione di m , o casuale (nel qual caso dev'essere trasmesso con m)
- La trasformata finale g è opzionale



Message Digest 5 (MD5)



- [RFC 1321], introdotto nel 1991, successore di MD4
- m di lunghezza qualsiasi, $n = 128$ bit, $q = 512$ bit
- Padding

Al messaggio originale viene aggiunto un blocco composto da:

- Un bit impostato a 1
- Una stringa di 0, lunga $m \bmod(512) - 1 - 64$
- Il numero $m \bmod(2^{64})$, codificato in 64 bit

Il risultato è una stringa di bit la cui dimensione è un multiplo intero t di 512 bit

- Funzione di compressione f

Lavora su blocchi di 512 bit ($q = 512$)

Composta da 4 *pass*: ciascun blocco da 512 bit viene processato da f in quattro distinte fasi

Produce un digest da 128 bit

- Molto veloce, ma:

Basato su una funzione (MD4) che è stata compromessa nel 1995 (sono state trovate collisioni)

Dal '96 ad oggi è stata dimostrata una crescente insicurezza di MD5. Es.: nel 2005 è stato dimostrato che bastano 8 ore ad un P4 a 1.6GHz per trovare due messaggi che collidono



Secure Hashing Algorithm 1 (SHA-1)



- [RFC 3174], introdotto nel 1993, basato su SHA, mai pubblicato (si pensa fosse vulnerabile)
- m di lunghezza minore di 2^{64} bit, $n = 160$, $q = 512$
- Padding: come per MD5 (senza $\text{mod}(2^{64})$)
- Funzione di compressione f
 - Lavora su blocchi di 512 bit ($q = 512$)
 - Composta da 5 pass: ciascun blocco da 512 bit viene processato da f in cinque distinte fasi
 - Produce un digest da 160 bit (riduce la probabilità di un *birthday attack*, rispetto a MD5)
- Decisamente più lento di MD5, ma più sicuro
- Altre versioni, derivate da SHA-1
 - SHA-256, SHA-384, and SHA-512
- Recentemente (estate 2005) ne è stata mostrata l'insicurezza
 - Bastano $O(2^{69})$ (l'ottimo sarebbe $O(2^{80})$) operazioni per trovare collisioni con SHA-1
 - La sicurezza di SHA-1 si sta "deteriorando" rapidamente...



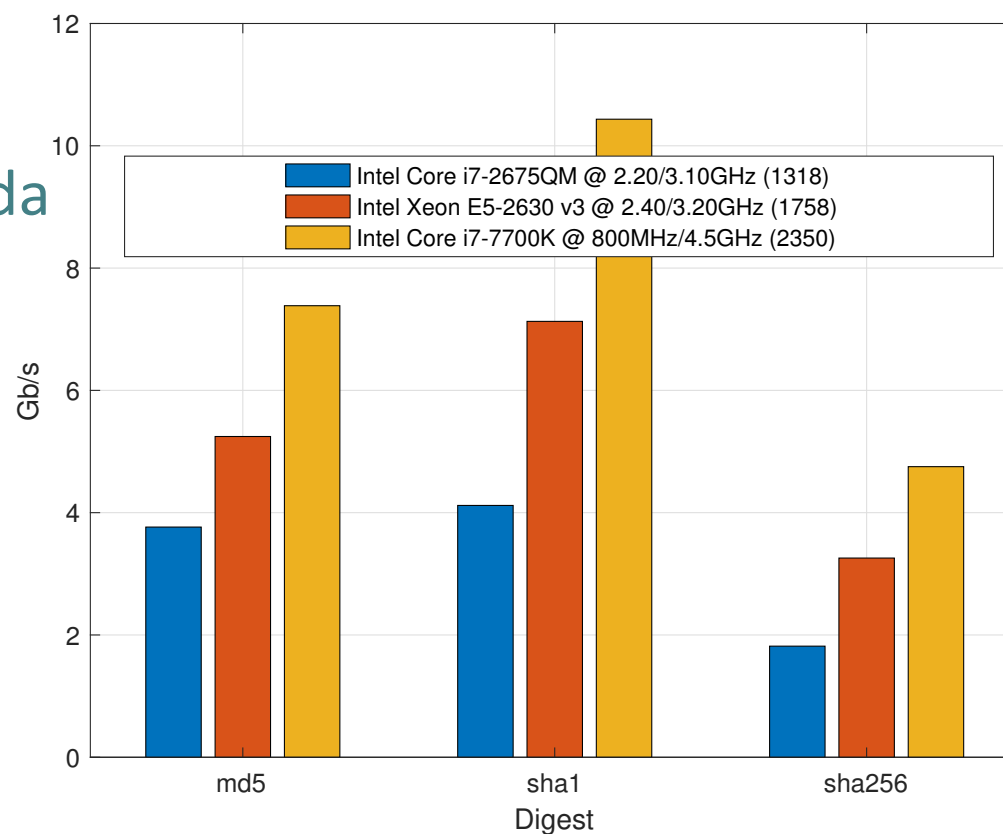
MDC: performance



- Risultati "in casa"

Creato grande file su
ramdisk

Risultato è una media da
molte esecuzioni

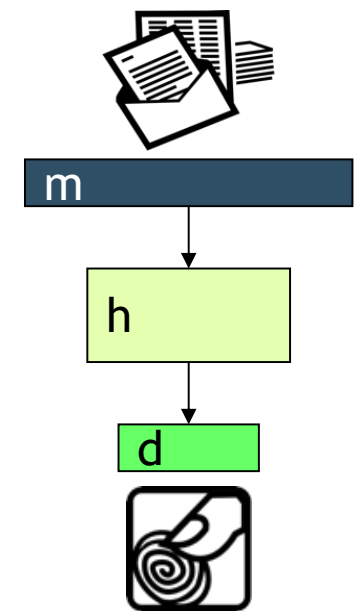


MDC:

proprietà fondamentali delle funzioni di hash



- Efficienza computazionale: dato m dev'essere semplice calcolare $d=h(m)$
- La probabilità che l'immagine attraverso h di un m scelto a caso sia un dato d (con $|D|=2^n$) deve essere 2^{-n}
Dato $d = h(m)$, il cambiamento di un solo bit di m deve portare al cambiamento, con probabilità 0.5, di tutti i bit di d
- Dato $d = h(m)$, dev'essere computazionalmente difficile risalire a m
Non invertibilità: *one-way hash*. Detta *preimage resistance*
- Dato un x t.c. $d=h(x)$, dev'essere computazionalmente difficile trovare $y \neq x$ t.c. $h(y)=d$
weak collision resistance, o *2nd preimage resistance*
- Deve essere computazionalmente difficile trovare due messaggi distinti che *collidano*
strong collision resistance
due messaggi x e y , con $x \neq y$, collidono se $h(x)=h(y)$
- **Nota** (*birthday problem*): dato n ($|D|=2^n$, n =lunghezza dell'hash), quanti messaggi a caso dovrò scegliere per avere probabilità 0.5 che i rispettivi hash collidano? Risposta: $2^{n/2}$
fate i conti con $n = 64...$



Obiettivi di sicurezza per un MDC

Note sulla *strong collision resistance*



- Obiettivo per un algoritmo crittografico a blocchi: fare in modo che l'attacco crittanalitico migliore sia una ricerca esaustiva nello spazio delle chiavi
- Obiettivo per un MDC (relativamente alla *strong collision resistance*): ***fare in modo che per trovare, con probabilità 0.5, due valori che producono lo stesso output si debbano calcolare $O(2^{n/2})$ hash***
 - n = dimensione in bit dell'hash
- Un MDC viene considerato sicuro se raggiunge questo obiettivo (insieme alla *preimage resistance*)



Derivazione di MAC a partire da MDC: un problema complesso



- I meccanismi più ovvi e intuitivi non funzionano (sono attaccabili facilmente). Sia $g()=MDC()$

- *Suffix-only*: $d = MAC_k(m) = g(m|k)$

Problema: conoscendo m e m' tali che $d=d'$, dove $d=g(m)$ e $d'=g(m')$ si sarà anche sicuri che $MAC_k(m)=MAC_k(m')$

- Il problema è dovuto che la k , per costruzione, viene usata solo nell'ultima iterazione della $f()$ usata dalla $g()$

Attacco in pratica: qualcuno vi fa "firmare" m , ed in seguito potrà sostenere che voi abbiate firmato m'

Il fatto di riuscire a montare un birthday attack sulla $g()$ porta a un attacco alla relativa $MAC_k()$ (e un birthday attack di questo tipo si può fare off-line, e senza conoscere k ...)

- *Prefix-only*: $d = MAC_k(m) = g(k|m)$

Problema: conoscendo $d=MAC_k(m)$ è semplice ricavare $d'=MAC_k(m|x)$, senza conoscere k

- Si può usare d come valore r_i nella f della particolare MDC usata...
- La cosa diventa più complicata con MDC che usano padding/lunghezza del messaggio, ma è sempre fattibile (in questo caso il messaggio falsificato non sarà $(m|x)$, bensì $(m|pad||x)$...)
- **Esercizio: trovare una soluzione (semplice) al problema**

Attacco in pratica: osservando un $d=MAC_k(m)$ è possibile generare un d' valido di una qualunque estensione di m

- *MD5 Prefix-suffix* (anche detto MD5 envelope): $d = MAC_k(m) = MD5(k | y | m | k)$

y = padding per portare k ad usare un blocco di 512 bit

Utile per fare in modo che la chiave entri direttamente in tutti i passi fatti dalla funzione di compressione f

Problema: in particolari condizioni, l'osservazione di sufficienti coppie $\langle d_i, m_i \rangle$ può portare a scoprire k



Hash-based MAC: HMAC



- [RFC 2104], (semplice) metodo dovuto a Bellare, Canetti e Krawczyk per utilizzare un MDC qualunque per ottenere un MAC (si parla di HMAC-MD5, HMAC-SHA1, ...)
- $(HMAC-MDC_k(m) =) MAC_k(m) = MDC((k \parallel 0...) \oplus c_1 \parallel MDC((k \parallel 0...) \oplus c_2) \parallel m))$
 - Nota: gli "0" rappresentano padding per portare k a una lunghezza pari alla lunghezza del blocco su cui opera MDC (512 bit per MD5 e SHA-1)
 - c_1 e c_2 sono due costanti
- Primo meccanismo di MAC basato su MDC per il quale si sia *dimostrata la sicurezza*, basandosi su ipotesi relativamente blande sull'MDC si partenza
 - L'applicazione di HMAC "risolve" anche i problemi di bassa collision-resistance di MD5
- Gli autori hanno dimostrato, tra le altre proprietà di HMAC, che esso è altrettanto sicuro di quanto è sicuro l'MDC sottostante. In breve, semplificando:
 - HP: MDC sia *collision-resistant*
 - TH: HMAC-MDC è sicura, ossia:
 - È collision-resistant: è computazionalmente impossibile trovare x e y , con $x \neq y$, tali che $HMAC-MDC_k(x) = HMAC-MDC_k(y)$
 - È computazionalmente impossibile trovare il corretto $HMAC-MDC_k(x)$, se non si conosce k , anche osservando un numero arbitrario di $HMAC-MDC_k(y)$, con $x \neq y$
- Prestazioni: praticamente le stesse della funzione di hash usata come base
- Morale: usate HMAC

